

Project 1

Pathfinding

Welcome To ML, Brother

9.22.2026

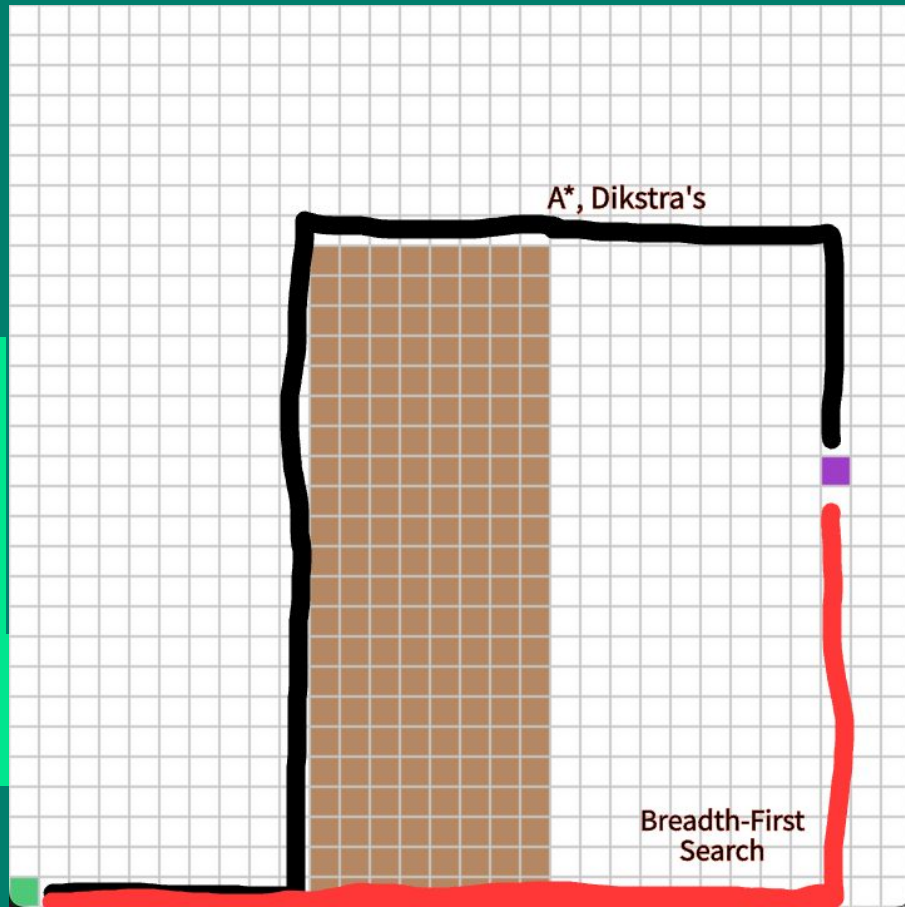
A*'s Evolution

BFS: searches everywhere, picks the oldest cell. Finds fewest steps, but is blind to cost.

Dijkstra: picks lowest g (cost to get to current point). Finds cheapest path, but searches everywhere.

A*: picks lowest $g + h$ (h = guess of cost left). Cheapest path, and searches less.

A* when $h = 0$ is Dijkstra's algorithm, used this formula in my code



u = u

Cell Types



Normal

cost = 1



Mud

cost = 5



Wall

cost = math.inf

Each cell has a cost value, used for the 'g' calculation for A* and Dijkstra's

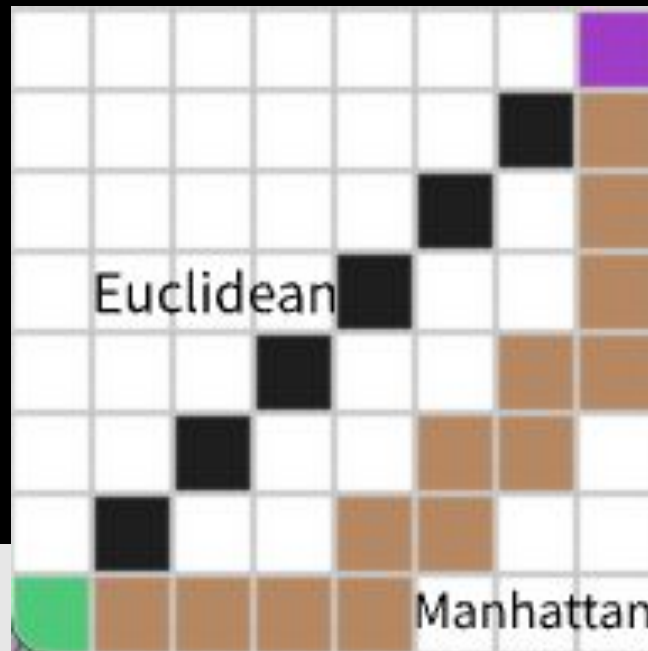
Since wall's cost is math.inf, a wall check would not be needed in astar.py. BFS does not look at costs, so the wall check is needed

Heuristic Choice

$h = \text{rows apart} + \text{columns apart}$

Manhattan: Restricted to just left-right-up-down movement, higher returning value than Euclidean but never higher than real value

Euclidean: Works if 8-directional movement (diagonals) are allowed, since h is lower, A* explores more cells



Code demonstration + unique
aspects

To use

Run `main.py` with Python 3.12 and PyGame installed (`> pip install pygame`).

The start is the green cell in the bottom-left corner.

- Click or click and drag to cycle cells: normal → mud → wall → normal
- Right click to move the goal
- `B / A / D / E` — pick BFS, A*, Dijkstra's, or Euclidean Heuristic, shown in the header
- `SPACE` — run the selected algorithm
- `R` — reset the search
- `M` — load the test maze
- `C` — clear the grid, reset back to starting grid

The terminal window prints the cost, steps, and cells explored after each successful run.

Test Results

Test maze (`H` key): a band of mud between the start and the goal, with a clear route above it.

Algorithm	Path cost	Steps	Cells explored
BFS	73	41	749
Dijkstra	57	57	796
A*	57	57	400

BFS took the fewest steps but the most expensive path, since it walked through 8 mud cells. Dijkstra and A* both found the cheapest path, but A* got there after exploring about half as many cells.

Unexpected: on the empty beginning grid, all algorithms explore all the cells. I think this is because every cell returns the same f value. Moving the goal anywhere else makes the A* algorithm best again

Limitations

- 4-directional movement only. Manhattan would overestimate with diagonal movement, so 8-d would need euclidean or Chebyshev instead.
 - Euclidean would be an overestimation - shown in demonstration how it is less efficient
- Fast mouse drags can skip cells, and when dragging, the transition from cell types can be confusing and fast

```
# Controls:
#   Left click / drag : cycle cells  normal (cost 1) -> mud (cost 5) -> wall -> normal
#   Right click      : place / move the goal cell
#   Start cell       : fixed at bottom-left (green)
#   B / A / D        : select BFS / A* / Dijkstra (only while idle)
#   SPACE            : run the selected algorithm
#   R                : reset the search (keeps the maze; unlocks editing)
#   C                : clear the maze (resets all cells to normal)
```

```
import math
import pygame
import bfs
import astar
from collections import deque
```

```
# ----- constants -----
CELL = 24 # pixel size of one square
COLS, ROWS = 30, 30 # grid dimensions
HEADER = 50 # pixels height for header
WIDTH = COLS * CELL
HEIGHT = ROWS * CELL + HEADER
```

```
COST_NORMAL = 1
COST_MUD = 5 # "costly" terrain
WALL = math.inf # a wall is just an infinitely expensive cell
STEPS_PER_FRAME = 3 # search speed: cells expanded per frame
```

```
# colors (R, G, B)
WHITE = (255, 255, 255) # normal cell
BROWN = (181, 136, 99) # costly cell ("mud")
BLACK = (30, 30, 30) # wall
GREEN = (80, 200, 120) # start
PURPLE = (160, 60, 200) # goal
GRID_LINE = (200, 200, 200)
BLUE = (70, 130, 220) # final path
```

Questions?